알약 탐지 및 분류 모델(Pill Detection) 프로젝트 보고서

1. 프로젝트 서론 (Introduction)

1) 프로젝트 개요 및 운영 배경

▼ **목표**

- 본 프로젝트는 모바일 앱에서 촬영된 경구약제 이미지를 입력으로 받아 '최대 4개의 알약 이름(클래스)과 위치(바운딩 박스)'를 고정밀 바운딩박스로 검출해내는 객체 탐지 모델을 개발하고, kaggle 평가 지표인 `mAP@[0.75:0.95]` 을 개선하는 것을 목표로 합니다.

▼ **접근 방식**

- 팀 협업에서 **재현성 · 비교 가능성 · 제출 안정성**을 확보하기 위해, GitHub 기반으로 설정/실험/산출물을 표준화한 **자동화 Base pipeline**을 구축하였습니다. 데이터 전처리부터 학습 · 평가 · 제출까지 절차를 고정하면, 팀원 간 환경 차이와 실수로 인한 성능 변동을 줄이고, 변경점이 성능에 미친 영향을 명확히 추적할 수 있을 것이라 판단했습니다.

▼ **초기 baseline 모델 구축**

- 빠르게 E2E 파이프라인을 세우는 목적으로, 현실적인 데이터 확인 및 제출 format을 확정할 수 있도록 구성하여 팀 공용 베이스라인을 잡기 위해 시도했습니다.

▼ **팀 공용 베이스라인 확정**

- E2E 파이프라인에서 확인한 결과를 기반으로 재현성과 데이터 정합성을 유지하기 위해, 성능. 속도, 안정성의 균형이 좋은 **YOLOv8s + imgsz 768 요건으로** 전처리–학습–추론–제출을 모두 표준화한 팀 공용 베이스라인을 확정하여 실험을 진행했습니다.

▼ **운영 과정**

- 전체 파이프라인은 run_pipeline.sh로 단일 명령 실행이 가능하도록 구성되어 있습니다.
- 레포지토리는 **STAGE 0~4 파이프라인**으로 구성되어 있으며, 각 단계별 산출물을 분리 저장해 재현성과 실험 비교 가능성을 확보할 수 있습니다.

▼ **자동화 pipeline STAGE 별 주요 작업**

- **STAGE 0** : 데이터 정제/검증/분할 + COCO 재조립
- **STAGE 1** : YOLO 학습용 데이터셋 변환
- **STAGE 2** : 모델 학습 (Ultralytics YOLO)
- **STAGE 3** : 모델 평가 (validation)
- **STAGE 4** : 테스트셋 추론 및 Kaggle 제출 CSV 생성/검증

2) 데이터셋 구성

- 기본 학습 데이터는 제공받은 **경구약제 이미지/어노테이션 데이터셋과,** 성능 개선을 위해 원본 데이터셋을 수집해 추가로 통합했습니다. 이 외부 데이터는 원본 학습 데이터와 **카테고리 ID 체계가 다를 수 있어 매핑을 적용**했습니다.
- 평가용 테스트 이미지는 **제공된 테스트 셋**으로 설정했습니다.

▼ 정제 전 (rawdata 전체)

💡
- 전체 원본 train 이미지: **14,244장**
  - 내부(train): **232장**
  - 외부(external): **14,012장**
  - 수동 제외 이미지: **208장**
- 전체 원본 test 이미지 : **842장**

▼ 데이터셋 위치 지정

- data/raw/train_images/ : 내부 학습 이미지
- data/raw/train_annotations/ : 내부 학습 어노테이션(COCO JSON)
- data/raw/test_images/ : 제출용 테스트 이미지
- data/raw/external/combined/images/ : 외부 데이터 이미지
- data/raw/external/combined/annotations/ : 외부 데이터 어노테이션

3) 평가 지표 설정 및 운영

- 대표 평가 지표로는 kaggle 평가 기준인 `mAP75_95`를 핵심 지표로 사용합니다.

▼ 이외 평가 지표 설정

💡 **1) 모델 성능 지표 (STAGE 2/3)**

- `mAP50` : IoU 0.50 기준 평균 정밀도(AP) 평균
- `mAP50_95` : IoU 0.50~0.95 구간 AP 평균
- `mAP75_95` : IoU 0.75~0.95 구간 AP 평균
- `precision` : 모델 예측 중 정답 비율
- `recall` : 실제 정답 중 검출한 비율
- `per_class_mAP75_95` : 클래스별 mAP75_95
- `per_class_ap50` : 클래스별 AP@0.5
- `per_class_ap50_95` : 클래스별 AP@[0.5:0.95]

2) 가중치 선택용 지표 (STAGE 2: competition_select)

- 비교 대상: best.pt, last.pt
- 우선 선택 기준: mAP75_95
- 예외 처리: 동점 또는 mAP75_95 미계산 시 후보 우선순위(설정 순서)로 선택

3) 제출 품질 검증 지표 (STAGE 4)

- total_rows제출 CSV 전체 행 수.
- n_images예측이 생성된 이미지 수.
- valid (PASS/FAIL)제출 형식/규칙 최종 통과 여부.
- n_errors, n_warnings검증 오류/경고 개수.

▼ **추가 검증 항목**
컬럼 스키마 일치, annotation_id 형식/중복,
image_id당 최대 박스 수(max_det_per_image) 위반 여부,
score 범위, bbox 값 유효성, category_id 유효 범위.

▼ **평가 운영 방식**

- **학습(STAGE 2) 단계**
  - 학습 결과로 생성된 best.pt와 last.pt를 **동일한 검증 셋에서 재평가**하고,

    competition_select.enabled: true 설정 시, `mAP75_95` 가 가장 높은 가중치를 competition_best.pt로 별도 저장합니다. 즉, 학습 중 선택된 “best”와 대회 지표 기준의 “best”를 분리해 관리합니다.
- **검증(STAGE 3) 단계**

- competition_best.pt(없으면 best.pt)로 평가를 수행하며, `mAP50`, `mAP50-95`, `mAP75_95`를 함께 기록해 지표 간 추이를 비교합니다.
- 평가 결과는 metrics.json에 `eval_mAP50`, `eval_mAP50_95`, `eval_mAP75_95`로 병합됩니다.

▼ 평가 결과 위치 지정

- 평가 결과는 metrics.json에 저장되고, 핵심 지표는 _registry.csv에 누적 기록하여 실험 간 성능 비교와 트래킹이 가능하도록 운영합니다.

4) 실험 환경 구축

- 실험은 Python 3.11.9 기반 가상환경(venv)에서 실행하며, 패키지 버전은 모두 requirements.txt로 고정해 재현성을 확보했습니다.

▼ 주요 패키지

▼ **딥러닝/학습 엔진**

- YOLO 학습/평가/추론을 CUDA 12.1 GPU 환경에 맞춰 고정.
- torch==2.5.1+cu121
- torchvision==0.20.1+cu121
- ultralytics==8.4.12
- ultralytics-thop==2.0.18

▼ **데이터 처리/분석**

- split, 통계, 전처리, 메트릭 분석 파이프라인용.
- numpy==2.3.5
- pandas==3.0.0
- polars==1.38.0
- scikit-learn==1.8.0
- scipy==1.17.0

▼ **이미지/증강**

- 이미지 로드/변환/증강, bbox 기반 전처리 및 시각화 처리.
- opencv-python==4.13.0.92
- opencv-python-headless==4.13.0.92
- albumentations==2.0.8
- albucore==0.0.24
- pillow==12.0.0

▼ **시각화/리포팅**

- 학습 곡선, 분포, 혼동행렬 등 결과 시각화.
- matplotlib==3.10.8
- seaborn==0.13.2

▼ **설정/입출력/유틸**

- YAML 기반 실험 설정 로딩, 검증, 외부 IO 보조.
- PyYAML==6.0.3
- pydantic==2.12.5
- requests==2.32.5

- 모든 실험 설정은 개인적으로 생성한 .yaml에서 관리하고, 실행 시 병합된 최종 설정을 config_resolved.yaml로 저장하여 재현성을 보장합니다.
- 실험 결과는 runs/, 제출 파일은 artifacts/submissions/, 우수 모델은 artifacts/best_models/에 정리하여 관리합니다.
- main develop 브랜치 기반으로 개별 브랜치를 생성하여 각자 작업 후, 사용할 코드는 develop에 merge합니다.

2. 데이터 탐색 및 전처리 (EDA & Preprocessing)

1) 클래스 분석 결과

▼ A. 클래스 분석 배경 및 필요성

- **Long-tail 현상 탐지**: 초기 EDA 결과, Kaggle 원본 데이터는 특정 클래스에 편중된 분포를 보였으며, 클래스당 평균 데이터가 13.6개로 딥러닝 모델의 일반화 성능을 기대하기 어려운 수준임을 확인하였습니다.
- **라벨링 결함 식별**: 외부 데이터 통합 과정에서 `category_id`가 모두 '1'로 기록된 중대한 데이터 결함을 발견하였으며, 이를 해결하기 위한 기술적 조치가 선행되었습니다.

▼ B. 기술적 성과: 스키마 역설계 및 라벨 복구

- 단순한 데이터 합치기를 넘어, 데이터의 신뢰성을 확보하기 위해 다음의 엔지니어링 기법을 적용하였습니다.
- **정규표현식(Regex) 기반 ID 복구**: JSON 메타데이터 내 `drug_N` 및 `dl_mapping_code` 필드에서 진짜 클래스 식별자를 추출하여, 오매핑된 라벨을 100% 정상화하였습니다.
- **다중 객체 낱개 파싱**: 이미지 1장 내 포함된 최대 4개의 알약을 개별 객체로 분리하여 데이터 가용성을 극대화하였습니다.

▼ C. 정량적 분석 결과 (Kaggle vs. External)

데이터 통합 전후의 지표 비교를 통해 확보된 학습셋의 우수성을 확인할 수 있습니다.

분석 지표 (Metric)	Kaggle (수집 전)	External (통합 후)	개선 효과
전체 객체(Box) 수	763개	**6,774개**	**약 8.8배 증분**
클래스당 평균 데이터	13.6개	**146.6개**	**약 1,070% 향상**
데이터 최소 보유량	3개	**9개**	**학습 하한선 상향**
신규 클래스 확보	-	**41종**	**탐지 범위 확장**

▼ D. 시각화 자료 분석

- **[Pink] Kaggle 원본**: 특정 클래스에 데이터가 전무하거나 극소수인 불균형 상태를 보여줍니다.
- **[Blue] External 보충**: Kaggle 데이터가 부족한 소수 클래스를 집중적으로 보완하여 전체적인 분포의 상향 평준화를 유도하였습니다.
- **[Total] 최종 통합**: 외부 데이터 기여율이 약 88.7%에 달하며, 모델이 모든 클래스에 대해 충분한 특징(Feature)을 학습할 수 있는 안정적인 환경이 구축되었음을 확인하였습니다.

2) 최종 데이터 정제 (STAGE 0)

▼ **STAGE 0의 목적**

- 데이터 정제는 STAGE 0 인 0_split_data.py에서 진행됩니다.
- STAGE 0의 목적은 원본 주석/이미지 데이터를 검증·정제해서 신뢰 가능한 학습 기준 데이터(df_clean.csv)를 만들고, 데이터 누수를 막는 group 기반 분할 정보(splits.csv)를 준비하는 것입니다. 즉, 다음 단계에서 사용할 수 있는 정제된 데이터를 만드는 단계입니다.

A. 데이터 정제 기준

▼ **정제 목록**

a. **수동 제외 목록 적용**

- exclude_4444_208.txt의 파일명을 우선 제외했습니다.
- 라벨 오류가 이미 확인된 샘플을 초기 단계에서 제거해 후속 단계로 노이즈가 전파되지 않도록 했습니다.

b. **JSON 구조 검증**

- images, annotations, categories가 1개씩 존재하는 split-COCO 형식만 허용했습니다.
- 구조 불일치 샘플은 정제 대상에서 제외했습니다.

c. **파일 정합성 검증**

- file_name 누락/공백 샘플 제외
- 동일 파일명이 여러 경로에 존재하는 이미지 파일명 중복(ambiguous) 케이스 제외
- 이미지-주석 매칭 불확실성을 원천 차단했습니다.

d. **외부 수집 데이터 이미지 메타 동기화**

- external 데이터는 이미지 파일 존재를 검증했습니다.
- external의 width/height는 실제 이미지에서 재측정해 동기화했습니다.
- 메타 불일치로 인한 bbox 정규화 오차를 줄였습니다.

e. **외부 수집 데이터 category 매핑 통일**

- 외부 데이터는 내부 데이터와 카테고리 ID 체계가 다를 수 있으므로 외부 데이터의 category를 내부 train 데이터 기준 canonical 체계로 매핑했습니다.
- 매핑 키는 drug_N 우선, 필요 시 fallback 필드를 사용했습니다.
- 결과적으로 train/external이 동일 class space를 공유하도록 맞췄습니다.

f. **bbox 유효성 검증**

- bbox 존재 여부, 길이 4, 숫자형 변환 가능 여부, w>0, h>0를 검사했습니다.
- 비정상 bbox는 제외 처리했습니다.

g. **OOB(경계 밖 bbox) 처리**

- 처리 정책: clip
- 이미지 경계를 넘는 bbox는 경계값으로 보정하고, 보정 후 면적이 0이면 제외했습니다.
- bbox 좌표 품질을 유지하면서 유효 샘플은 최대한 보존했습니다.

h. **그룹 기반 분할 준비**

- 파일명에서 group_id를 추출(K-\d+-\d+-\d+)했습니다.
- 동일 계열 샘플이 train/val에 동시에 들어가지 않도록 split 단위를 group으로 통일했습니다.

▼ **정제 output 파일 목록**

💡 **정제/분할 산출물**

- data/processed/cache/<run_name>/ : STAGE 0 정제 결과
- data/metadata/ : 분할/감사 로그 및 매핑 테이블

▼ **주요 산출물 파일**

- df_clean.csv : 정제된 객체 레코드
- splits.csv : train/val 분할 결과
- train_merged_coco.json : 정제 후 COCO 통합본
- label_map_full.json : 카테고리 ID ↔ 인덱스 매핑

B. 정제 결과 데이터셋

▼ **정제 데이터셋 구성**

💡

- 정제 후 학습 시 사용 이미지: 10,226
- 정제 후 학습 시 사용 객체(박스) 수: 39,226

3) 최종 이미지 전처리 (STAGE 1)

▼ **STAGE 1의 목적**

STAGE 0에서 만든 정제 데이터(df_clean.csv, splits.csv)를 받아서, YOLO 학습에 쓸 수 있는 형식으로 변환하는 것입니다. 즉, 이미지/라벨을 train/val 폴더 구조로 배치하고, bbox를 YOLO 정규화 좌표로 변환하며, data.yaml / class_map.csv 를 생성해 학습할 데이터 포맷을 완성하는 단계입니다.

▼ **YOLO 학습 포맷 변환 과정**

▼ **기본 로직 구조**

1_preprocess.py가 export_yolo.py를 호출해 df_clean/splits를 YOLO 디렉터리와 data.yaml로 내보내는 구조입니다.

A. 입력

- 정제 어노테이션: df_clean.csv
- 학습/검증 분할표: splits.csv
- 원본 이미지 루트: data/raw/train_images
- 외부 이미지 루트: data/raw/external/combined/images
- 실행 엔트리: 1_preprocess.py
- 핵심 변환 모듈: export_yolo.py

B. 처리 절차

a. 입력 파일 존재 여부 및 필수 컬럼 검증

b. category_id를 고유값 정렬 후 YOLO용 class_index(0 ~ nc-1)로 재매핑

c. group_id 기준으로 train/val split 적용

이미지 파일을 hardlink 우선으로 연결(실패 시 copy fallback 가능)

d. bbox(x,y,w,h)를 **YOLO 정규화 좌표**(xc,yc,w,h, 0~1)로 변환

- 입력: COCO형 bbox_x, bbox_y, bbox_w, bbox_h + 이미지 width,height
- 변환: xc,yc,w,h를 0~1 정규화
- 라벨 한 줄 형식: class_index xc yc w h
- 이상값은 skip/clip(클램프) 통계로 기록

e. 이미지별 .txt 라벨 생성ㄴ

f. 산출물 생성

data.yaml, (path/train/val/nc/names)

class_map.csv,

convert_manifest.json (누락 이미지 수, invalid bbox 수 등)

g. 라벨 파일 재검증(형식/숫자/범위)

C. Output

출력 루트: data/processed/datasets/<dataset_prefix>_<run_name>/

- images/train, images/val : train으로 배정된 이미지 / val로 배정된 이미지 분배
- labels/train, labels/val : 위의 각 이미지와 1:1 대응되는 YOLO 라벨(.txt) 폴더

- data.yaml : YOLO 학습 진입 파일
- convert_manifest.json : STAGE1 변환 요약 리포트. 품질 통계 기록
- 추가 메타: class_map.csv : 원본 category_id를 YOLO 학습용 연속 인덱스 (class_index)로 매핑한 테이블. 학습 결과 해석이나 제출 후처리 시 클래스 역매핑 기준으로 사용

4) 데이터 증강(Augmentation) 조건

- **Baseline 기본값 + 실험 설계에 따른 조건별 차별화**

5) 데이터 로더 구성

▼ **개요**

본 파이프라인에서 데이터 분할과 데이터 로더 구성은 STAGE 0와 STAGE 1에서 확정되며, STAGE 2(학습)와 STAGE 3(평가)는 이 산출물을 그대로 사용한다.

▼ **Train/Valid/Test Split 설계**

a. **Train/Valid 분할 주체**
   - 분할은 0_split_data.py에서 수행됩니다.

b. **분할 비율**
   - 기본 설정은 train_ratio: 0.8로, Train 80% / Valid 20% 구조입니다.
   - 기준 설정: base.yaml

c. **누수 방지 방식**
   - 단순 랜덤 분할이 아니라 group_id 기반 분할을 적용합니다.
   - 목적은 동일 계열 샘플이 Train과 Valid에 동시에 들어가 성능이 과대평가되는 문제를 줄이는 것입니다.
   - 기본 규칙:
     - 정규식: K-\d+-\d+-\d+
     - fallback: prefix_before_underscore

d. **분할 결과 기록**
   - 메타 분할 결과는 splits.csv에 저장됩니다.
   - 이후 STAGE 1에서 YOLO용 폴더/라벨 구조로 변환되며 최종 데이터 정의는 data.yaml로 고정됩니다.

e. **Test dataset 처리 방식**
   - Test는 Train/Valid처럼 분할하지 않습니다.
   - STAGE 4에서 data/raw/test_images 전체에 대해 최종 추론을 수행하게 됩니다.

3. 모델 설계 및 학습 (Modeling & Training)

초기 Baseline모델 설계 로직

1) Baseline모델 전체 아키텍처 설계 (STAGE 2)

▼ **기준 설정 구조**

- Baseline 모델은 총 3가지의 YOLO 기반 기본 모델학습 설계 중

  가장 kaggle성능이 뛰어난 점수를 얻은 모델로 선정되었습니다.

- 기본 구조는 base.yaml을 _base_로 상속하되,

  실험 yaml 파일에서 명시한 값이 우선 적용됩니다.

  적용 로직: config_loader.py

▼ **YOLOv8s 기본 모델 설정 이유**

- **빠른 학습 및 추론 가능성**: 팀원이 여러 번 실험해야 하므로 iteration 비용이 낮아야 합니다.
- **좋은 성능** : 개선의 기준이 될 만큼 충분한 점수가 나와야 합니다.
- **재현/관리 용이성**: Ultralytics 생태계(학습 로그, best/last, 결과 csv)가 표준화되어 있습니다.
- **작은 데이터에서의 과적합 리스크**: 232장 규모의 작은 데이터에서 너무 큰 모델은 불안정성이 높을 수 있다고 판단했습니다.
- 위 사항의 적절한 균형점이 **YOLOv8s이었으며,** kaggle Public score가 여러 추론 실험에서도 크게 흔들리지 않는 안정성을 보여주었기 때문에 이를 기본 모델로 선정하였습니다.

A. 아키텍처 정의

▼ **아키텍처 메타 정보**

- model.architecture: "yolov8s"

  결과 레지스트리 기록용으로 사용됩니다.

- 실제 로딩에 직접 쓰이는 값은 아니고, 실험 추적용 메타데이터 성격입니다.

▼ **실제 로딩 모델**

- model.pretrained: "yolov8s.pt"

  PillDetector.from_config()가 이 값을 사용해 **Ultralytics YOLO 모델을 로드**합니다.

  즉, **YOLOv8s 사전학습 가중치 기반 전이학습**이 베이스라인 전략입니다.

B. 학습 전략 개요

▼ **전이학습(Transfer Learning)**

- 사전학습 가중치(yolov8s.pt)에서 시작해 대회 데이터에 맞게 fine-tuning합니다.

- pretrained: true로 설정되어 있어, Ultralytics 내부에서 사전학습 사용을 명시합니다.

▼ **외부 데이터 사용 정책**

- external_data.enabled: false

  외부 데이터 통합을 비활성화합니다. (Baseline 기준 → 추후 성능 개선을 위해 True로 변경)

  즉, **원본 train 데이터만으로 학습**합니다.

  (base.yaml의 기본값 true를 이 실험 설정이 명시적으로 끕니다.)

C. 데이터 증강 전략 (기본값)

▼ **개요**

- 이 데이터 증강 전략은 **Baseline에 지정된 기본값**에 관한 설명이며,

  최종 제출 모델에서는 앞의 **"2. 데이터 탐색 및 전처리"** 파트에서 설명하였다시피

  실험 수행 요건에 따라 **수정 및 개편됨.**

▼ **색상/밝기 계열**

- **hsv_h**: 0.015
- **hsv_s**: 0.7
- **hsv_v**: 0.4

▼ **기하학적 변환**

- degrees: 0.0
- **translate**: 0.1
- **scale**: 0.5
- shear: 0.0
- perspective: 0.0

▼ **좌우/상하 반전**

- flipud: 0.0
- **fliplr**: 0.5

▼ **조합 증강**

- **mosaic**: 1.0 (강하게 사용)
- mixup: 0.0
- copy_paste: 0.0

▼ **요약**

- **Mosaic 적극 사용**, **MixUp/Copy-Paste 미사용**,
- **색상/밝기 증강 + 스케일/이동 변환**으로 데이터 다양성을 확보하는 전략입니다.

D. 주요 학습 설정 (하이퍼파라미터 기본값)

▼ **에폭/해상도/배치**

- epochs: 100
- imgsz: 768
- batch: 8

  해상도를 768로 올리고 배치를 8로 유지하는 구성

▼ **Optimizer 및 학습률 정책**

- optimizer: "auto"

  Ultralytics가 모델/배치 상황에 따라 최적화를 자동 선택
- cos_lr: false

  코사인 스케줄링을 사용하지 않고 **기본 LR 스케줄**을 따릅니다.
- close_mosaic: 10

  마지막 10 epoch에서 mosaic 증강을 끄는 전략

▼ **조기 종료/재현성**

- patience: 30

  개선 정체 시 30 epoch까지 기다린 뒤 조기 종료.
- seed: 42, deterministic: true

  재현성 확보를 위한 결정론적 실행을 목표로 합니다.

▼ **자원 및 실행 환경**

- device: 0

  기본적으로 GPU 0 사용(명시적 지정).
- workers: 4

  데이터 로딩 워커 4개 사용.

E. 체크포인트 및 기록 정책

▼ **주요 정책**

- save: true, save_period: -1

  중간 저장 주기는 사용하지 않고, **best/last 중심으로 저장**합니다.
- plots: true

  학습/검증 곡선 및 시각화 결과 저장.
- debug_snapshots.enabled: true

  학습 중 **val 배치 스냅샷 저장** 기능을 활성화합니다.

(스냅샷 상세 로직은 detector.py에서 수행)

2) Baseline모델 Loss Function & Optimizer 설계

A. Loss 구성 요소 & 가중치 설정

▼ **Loss 구성 및 가중치 명시 여부**

- Baseline 파이프라인은 **Ultralytics YOLO의 기본 손실 구조**

  (박스 회귀(box loss) + 분류(class loss) + DFL(Distribution Focal Loss) 를 사용합니다.
- PillDetector.train()에서 train 섹션의 box, cls, dfl 값을 Ultralytics 인자로 직접 전달합니다.
- Baseline model 학습용 기본값인 yolov8s_baseline.yaml에는

  별도 loss 가중치가 **명시되지 않으므로**, base.yaml에 설정한 기본값을 적용합니다.

  ▼ **base.yaml 기본 적용값**

  - **Box loss**: 7.5
  - **Class loss**: 0.5
  - **DFL loss**(Distribution Focal Loss): 1.5
- **Label smoothing**: 0.0

  라벨 스무딩은 비활성 상태입니다. (기본값)

B. Optimizer 설정

▼ **Optimizer 설정값**

- **optimizer**: "auto"

  Ultralytics가 내부 규칙에 따라 **최적화 알고리즘을 자동 선택**합니다.

  (SGD/Adam/AdamW 중 자동 결정)
- 학습률 관련 기본값은 base.yaml을 따릅니다:
  - **lr0**: 0.01 (초기 학습률)
  - **lrf**: 0.01 (최종 학습률 비율, lr0 * lrf)
- 스케줄링:
  - **cos_lr**: false

    코사인 스케줄러는 사용하지 않습니다. 즉, **기본 학습률 스케줄**을 따릅니다.

▼ **요약**

Baseline 모델은 Ultralytics YOLOv8 기본 손실 구조(Box/Cls/DFL)를 사용하며, 손실 가중치는 box=7.5, cls=0.5, dfl=1.5로 설정하였습니다. Label smoothing은 적용하지 않았습니다. Optimizer는 Ultralytics의 auto 모드를 사용하여 모델/환경에 맞는 기본 최적화기를 자동 선택하도록 구성하였습니다.

3) Baseline모델 학습 루프 및 모니터링 설계

A. 학습 루프 실행 흐름 및 경로

▼ **학습 루프 흐름**

- **모델 학습 진행 경로**

  모델 학습은 run_pipeline.sh 중 **STAGE 2**에 해당하며, 2_train.py 에서 진행

- **STAGE2 루프 실행**

  실험 config를 로드하고 PillDetector.train() 호출로 Ultralytics 학습 루프를 실행합니다.

  - config는 load_experiment_config()로 읽고 병합됩니다.
  - 이후 PillDetector.from_config() 또는 PillDetector.from_weights()로 모델을 준비합니다.
  - 최종적으로 detector.train(...) 호출로 **Ultralytics 학습 루프가 시작**됩니다.
  - 학습 결과는 runs/<run_name>/train 에 저장됩니다.

    paths.artifact_layout: compact → 즉, 실행 결과가 **한 폴더 안에 모입니다.** (base.yaml 기본값)

    근거: detector.py

B. Ultralytics 학습 호출

▼ **Ultralytics 호출**

- PillDetector.train()은 train 섹션을 Ultralytics model.train() 인자로 직접 전달합니다.
- exist_ok=True

  동일 이름의 실험 폴더는 덮어씁니다.

C. 학습 로그 출력 과정

▼ **로그 출력 관련 설정**

- log_mode: "batch"

  기본적으로 Ultralytics의 **배치 진행 로그/프로그레스바를 그대로 사용**합니다.

- verbose: true

  상세 로그 모드로 출력

- log_mode가 "epoch"일 경우 배치 로그는 줄이고
  에폭 요약 로그를 별도로 찍도록 설계되어 있으나,
  기본 설정은 "batch"이므로 에폭 요약 로그 콜백은 비활성 상태입니다.

D. 모니터링 지표(학습 곡선) 기록 과정

▼ 학습 곡선 기록

- 학습 중 Ultralytics 기본 로그 및 results.csv가 생성됩니다.
- **plots**: true
  학습 곡선 및 검증 이미지가 자동 생성됩니다.
- **학습 곡선 기록 과정**

1. **STAGE 2**
   - run_pipeline.sh가 STAGE 2에서 python scripts/2_train.py --run-name YOLOv8s_baseline --config ...를 호출합니다.
   - 이후 학습 로그 및 곡선 생성이 시작됩니다.
2. **출력 경로 결정 (compact 레이아웃)**
   - paths.artifact_layout가 기본 compact이므로,
     학습 결과 (**학습 곡선 및 시각화 산출물) 저장** 경로는
     runs/YOLOv8s_baseline/train로 고정됩니다.
3. **Ultralytics 학습 루프 실행**
   - 2_train.py → PillDetector.train() → self.model.train(**kwargs) 흐름으로 Ultralytics YOLO 학습 루프가 돌기 시작합니다.
   - 이때 Ultralytics가 **에폭 단위로 metrics를 기록**합니다.
4. **학습 곡선 원본 로그: results.csv**
   - Ultralytics는 매 epoch의 손실/지표를 results.csv에 기록합니다.
   - **이 CSV가 모든 곡선의 원본 데이터**입니다.
5. **학습 곡선 이미지 자동 생성**

- train.plots: true
  학습 종료 시 자동으로 시각화 파일들이 생성됩니다.
- 저장 위치: runs/YOLOv8s_baseline/train/
- 주요 이미지 파일:
  - results.png : 학습/검증 loss + mAP/Precision/Recall 기록된 종합 곡선 그래프
  - BoxF1_curve.png, BoxP_curve.png, BoxPR_curve.png, BoxR_curve.png
  - confusion_matrix.png, confusion_matrix_normalized.png
  - labels.jpg

E. 스냅샷 기반 모니터링(시각적 디버깅) 과정

▼ 스냅샷 모니터링 설정

- debug_snapshots.enabled: true

  학습 중 val 배치 결과 이미지를 n개 시점에서 주기적으로 복사해 자동으로 저장합니다.
- 스냅샷 저장 시점은 base.yaml 기준 규칙을 따릅니다:
  - 퍼센트 포인트: [10, 30, 50, 70, 90, 100]

    - 최대 간격 보정: max_epoch_gap: 20
    - 저장 폴더: runs/<run_name>/train/snapshots
- 이 스냅샷은 학습 진행 상태 및 품질을 **시각적으로 점검**하는 용도입니다.

F. 학습 종료 후 자동 후처리 과정

▼ **후처리 작업 수행 종류**

▼ **현 baseline에도 설정이 유지된 후처리 작업**

- best.pt와 last.pt를 확인해 **best 가중치 복사/보관**을 수행합니다.
- metrics.json 생성:
    - **mAP50, mAP50-95, mAP75-95(대회 지표) 저장**
- 레지스트리 _registry.csv 갱신:
    - 수행한 **학습 모델/하이퍼파라미터/성능/가중치 경로가 기록**됩니다.

▼ **초기 baseline 설정에 추가되었지만 현재는 빠진 작업**

- **제출 후보 다중 생성(conf sweep)**:
    - conf = 0.001, 0.01, 0.02, 0.03, 0.05로 submission CSV 일괄 생성
- **제출 후보 다중 생성(iou/imgsz sweep)**:
    - iou = 0.4/0.5/0.6, imgsz = 768/896/1024 조합별 CSV 생성
- **TTA 제출 후보 생성**:
    - augment=True 추론으로 _tta.csv 별도 생성
- **raw 예측 캐시 기반 오프라인 후처리**:
    - pred_test_cache_full_*.jsonl 저장 후 conf/topk 재적용으로 재추론 없이 후보 생성
- **raw max_det 상향 적용**:
    - topk 전 후보 손실 방지를 위해 raw max_det를 크게 사용

▼ **요약**

학습 루프는 Ultralytics YOLO의 기본 train 루프를 그대로 사용했으며, 별도의 커스텀 루프나 WandB/TensorBoard 연동은 적용하지 않았다. 대신 Ultralytics가 자동 생성하는 results.csv와 각종 곡선/시각화 파일을 통해 학습 경과와 성능을 모니터링하였다.

▼ 주요 모니터링 시각화 산출물

- results.csv
    - epoch별 학습/검증 메트릭 기록 (mAP, precision, recall, loss 등)
- results.png

    - 학습 곡선 요약 그래프 (loss / metric trend)
- BoxPR_curve.png, BoxF1_curve.png, BoxP_curve.png, BoxR_curve.png
    - PR/F1/P/R 곡선 시각화
- confusion_matrix.png, confusion_matrix_normalized.png
    - 클래스 혼동 행렬
- val_batch*_pred.jpg, val_batch*_labels.jpg
    - 검증 샘플 예측 시각화 (정성 평가용)

4) Baseline모델 성능 최적화 기술 설계

▼ A. 입력 해상도 상향 (imgsz=768)

- Base 기본값 640보다 큰 해상도를 사용합니다.

  → 알약처럼 작은 객체가 많은 데이터에서 **세부 패턴/경계 인식 향상**을 기대한 선택.

▼ B. Mosaic 강하게 사용 (mosaic=1.0) + MixUp/Copy-Paste 비활성

- Mosaic는 데이터 다양성 및 배경/스케일 변화에 강점이 있습니다.

  → **일반화 성능 개선 목적으로 사용**

▼ C. 색상/밝기 증강 활성

- HSV 증강(hsv_h/s/v)과 스케일/이동 변환(translate, scale)을 활성화했습니다.

  → 촬영 환경(조명, 색온도, 배경 등) 변화에 대한 **강건성 확보**.

▼ D. close_mosaic=10

- 학습 후반 10 epoch에서 Mosaic를 끊습니다.

  → 마지막 단계에서 **실제 분포에 더 가까운 데이터로 미세 조정**해 성능을 안정화하기 위한 전략.

▼ E. 결정론적 학습 + 고정 시드

- **deterministic** : true, **seed** : 42

  → 실험 재현성 확보 및 결과 비교 신뢰도 향상.

▼ F. Early Stopping (patience=30)

- 충분히 기다리되 성능 정체 시 학습을 종료해 **과적합 및 자원 낭비를 줄이는 목적**.

▼ 요약

본 베이스라인에서는 고급 최적화는 적용하지 않았습니다. Ultralytics YOLOv8s 기본 구조를 사용하고, image 사이즈 상향과 강한 mosaic로 작은 객체 인식과 일반화를 노리고, 색상·기하 증강 적용으로 촬영 환경 변화에 강건하게 만들었으며, 후반에는 mosaic를 끄고 미세 조정하였습니다.

재현성 확보를 위해 고정 시드/결정론을 쓰고, early stopping으로 과적합을 줄이며, 스냅샷으로 학습 중 오류 패턴을 빠르게 확인하도록 구성했습니다.

5) 초기 Baseline모델 학습 및 평가 결과

▼ **학습 및 평가 결과 지표** (metrics.json 기준)

▼ **Train set 학습 결과**

- mAP50: **0.966**
- mAP50_95: **0.952**
- mAP75_95: **0.9387**
- Precision: **0.9317**
- Recall: **0.9627**

▼ **학습 그래프**

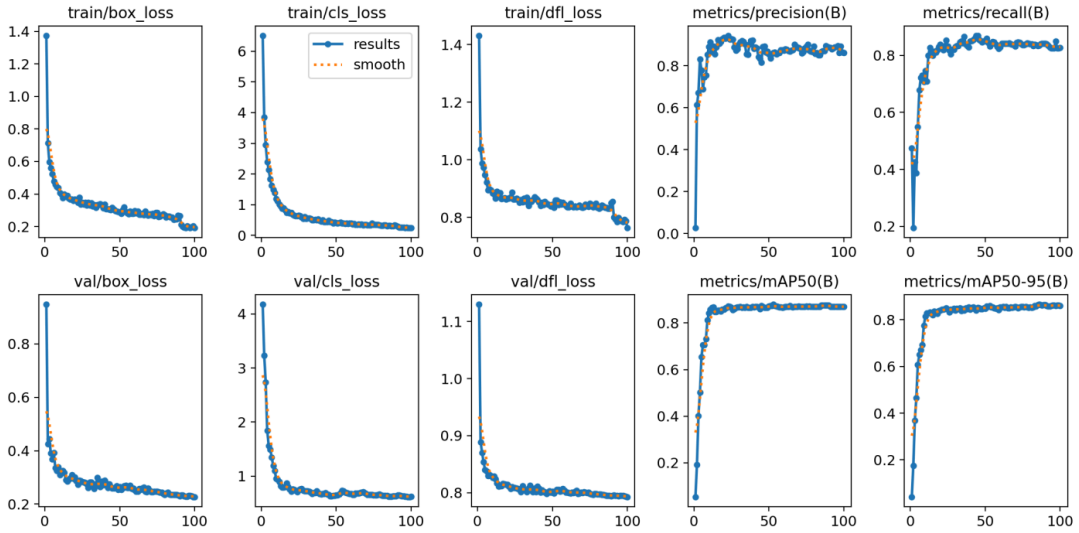


Figure 1. YOLOv8s Baseline 학습/검증 성능 추이 (Epoch별 Loss 및 Detection Metrics)

▼ **Figure 1 설명**

- train/box_loss

  x축은 epoch, y축은 train box loss(학습 데이터의 박스 위치 회귀 오차)입니다. 값이 낮을수록 예측 박스 좌표가 정답 박스에 가깝다는 의미입니다. 본 실험에서는 초반 급감 후 완만히 감소하여 위치 학습이 안정적으로 수렴했습니다.

- train/cls_loss

  x축은 epoch, y축은 train cls loss(학습 데이터의 클래스 분류 오차)입니다. 값이 낮을수록 분류가 정확합니다. 초반 큰 폭으로 감소하고 이후 완만히 줄어들어 클래스 판별 성능이 빠르게 개선된 뒤 안정화되었습니다.

- train/dfl_loss

  x축은 epoch, y축은 train dfl loss(Distribution Focal Loss 기반 박스 정밀도 오차)입니다. 값이 낮을수록 박스 경계 품질이 좋습니다. 초반 하락 후 점진적으로 감소하며 박스 품질이 꾸준히 개선되었습니다.

- metrics/precision(B)

  x축은 epoch, y축은 precision(검증 시 예측한 객체 중 정답 비율)입니다. 값이 높을수록 오탐(False Positive)이 적습니다. 초반 급상승 후 0.85~0.9 부근에서 유지되어 정밀도가 안정적으로 확보되었습니다.

- metrics/recall(B)

  x축은 epoch, y축은 recall(정답 객체 중 검출한 비율)입니다. 값이 높을수록 미탐(False Negative)이 적습니다. 초반 빠르게 상승한 뒤 0.8대에서 안정화되어 객체 누락이 크게 줄었습니다.

- val/box_loss

  x축은 epoch, y축은 val box loss(검증 데이터의 박스 위치 오차)입니다. 값이 낮을수록 일반화된 위치 예측 성능이 좋습니다. 학습 손실과 함께 지속적으로 감소하여 검증셋에서도 위치 추정 성능이 개선되었습니다.

- val/cls_loss

  x축은 epoch, y축은 val cls loss(검증 데이터의 분류 오차)입니다. 값이 낮을수록 검증셋 분류 성능이 좋습니다. 초반 급감 후 안정적으로 낮아져 과도한 분류 과적합 없이 수렴했습니다.

- val/dfl_loss

  x축은 epoch, y축은 val dfl loss(검증 데이터 박스 정밀도 오차)입니다. 값이 낮을수록 검증셋 박스 품질이 좋습니다. 꾸준한 감소 추세로 박스 일반화 성능이 개선되었습니다.

- metrics/mAP50(B)

  x축은 epoch, y축은 mAP50(IoU 0.50 기준 평균 정밀도)입니다. 값이 높을수록 탐지 성능이 좋습니다. 초반 급상승 후 0.87 수준에서 plateau를 형성해 조기 구간에서 주요 성능을 확보했습니다.

- metrics/mAP50-95(B)

  x축은 epoch, y축은 mAP50-95(IoU 0.50~0.95 평균 AP, 더 엄격한 종합 지표)입니다. 값이 높을수록 박스 위치와 분류를 포함한 전체 탐지 품질이 우수합니다. 빠르게 상승 후 0.86 내외로 안정화되어 전반적 검출 품질이 높게 유지되었습니다.

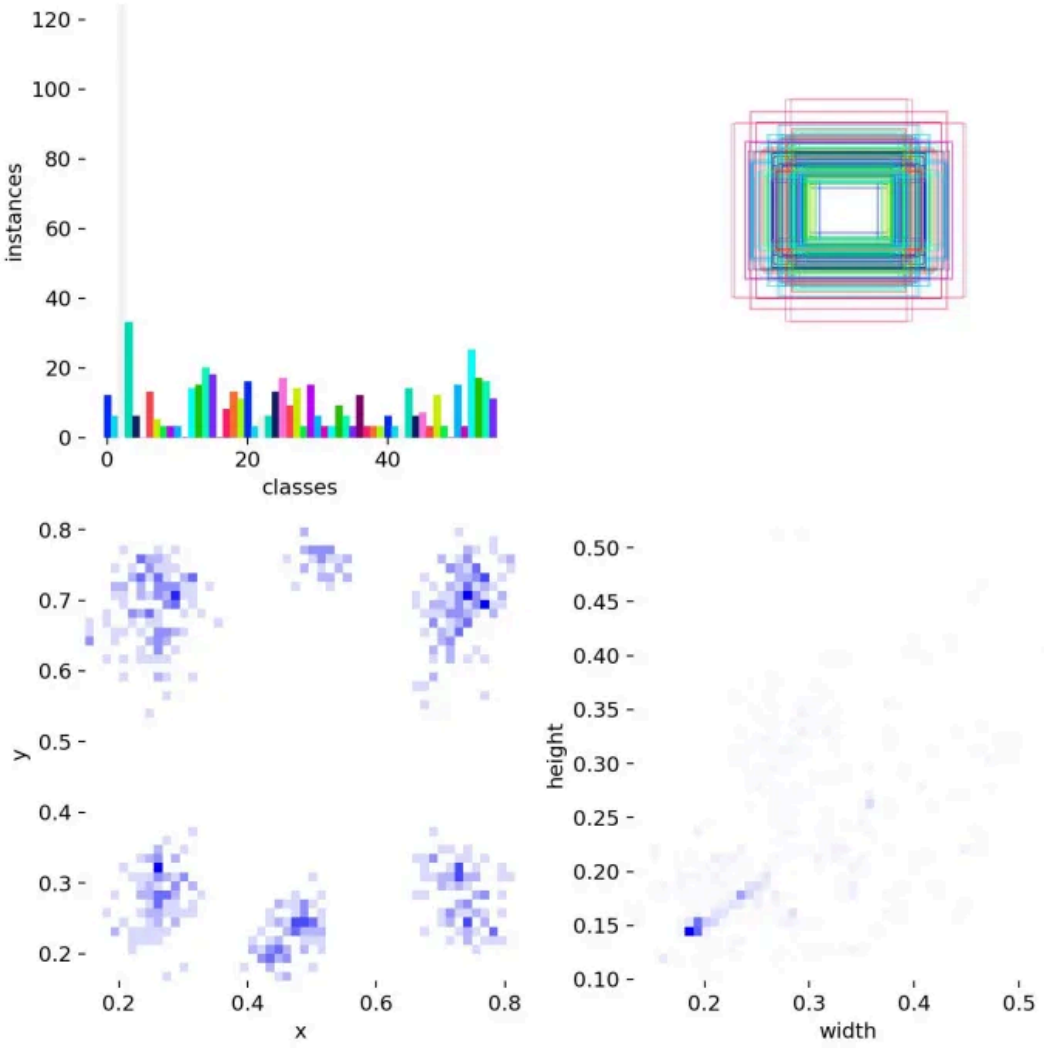


Figure 2. YOLOv8s Baseline 학습 데이터 라벨 분포 (클래스 빈도, bbox 위치·크기 분포)

▼ Figure 2 설명

- 좌상단 (classes vs instances)

  각 클래스별 라벨 개수(인스턴스 수) 분포입니다.

  현재 그래프는 클래스별 개수가 균일하지 않고 차이가 있습니다. 즉, 일부 클래스에 데이터가 더 많고 일부는 상대적으로 적은 **클래스 불균형**이 있습니다.

- 우상단 (겹쳐진 사각형들)

  bbox의 폭/높이 분포를 같은 중심에 겹쳐 그린 도식입니다.

  사각형이 한 구간에 많이 겹쳐 있어, 박스 크기가 완전히 랜덤하기보다 **특정 크기 범위에 집중**되어 있습니다. 극단적으로 큰/작은 박스는 상대적으로 적어 보입니다.

- 좌하단 (x vs y)

  bbox 중심점(center)의 위치 분포입니다.

점들이 몇 개의 클러스터로 모여 있어 객체가 이미지 전체에 균일하게 퍼지기보다 **특정 위치대에 자주 등장**하는 경향이 있습니다(위쪽/아래쪽 일부 구간에 밀집).

- 우하단 (width vs height)

  bbox 너비-높이 분포입니다.

  대각선 방향 패턴이 보여 너비가 커질수록 높이도 함께 커지는 경향(비율 유사)이 있습니다. 동시에 특정 구간에 밀집되어 있어 **유사한 스케일/종횡비의 객체가 많이 포함**된 데이터셋으로 해석됩니다.

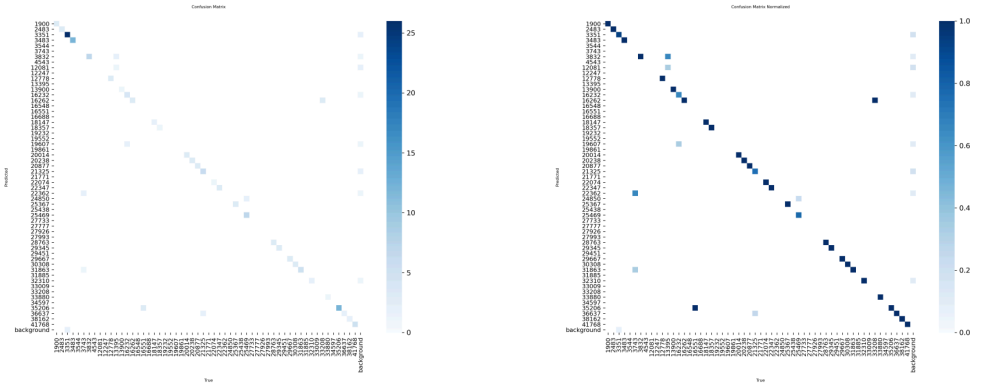


Figure 3. YOLOv8s Baseline 학습 단계 혼동행렬 분석 (Raw Count / Normalized)

▼ **Figure 3 설명**

- **왼쪽 Confusion Matrix** (원시 카운트; 즉, 절대적 개수 관점)
  - x축은 실제 클래스(True), y축은 예측 클래스(Predicted)입니다.
  - 대각선이 진할수록 정분류 개수가 많다는 의미입니다.
  - 현재 그래프는 대각선에 값이 많이 모여 있어 전반적으로 클래스 구분이 잘 되고 있습니다.
  - 일부 대각선 밖 점(오분류)이 보이며, 특정 클래스끼리 혼동이 발생하고 있습니다.
  - 마지막 background 축 부근 값은 미탐/오탐과 관련된 오류가 일부 존재함을 보여줍니다.
  - 색이 아주 진한 클래스와 옅은 클래스가 공존하므로, 클래스별 샘플 수(지원 수) 차이가 있다는 점도 확인됩니다.

- **오른쪽 Confusion Matrix Normalized** (정규화 비율; 즉, 클래스별 상대적 성능 관점)
  - 같은 행렬을 0~1 비율로 정규화한 그래프입니다.
  - 각 칸의 값은 "해당 실제 클래스에서 어느 클래스로 예측되었는지"의 비율로 해석합니다.

- 대각선의 진한 값(1에 가까움)은 그 클래스 재현율이 높다는 뜻입니다.
- 대각선 밖 진한 점은 특정 클래스 간 혼동 비율이 상대적으로 크다는 의미입니다.
- 원시 카운트와 달리 클래스 개수 차이 영향을 줄여서, 어떤 클래스가 구조적으로 어려운지 비교하기에 더 적합합니다.

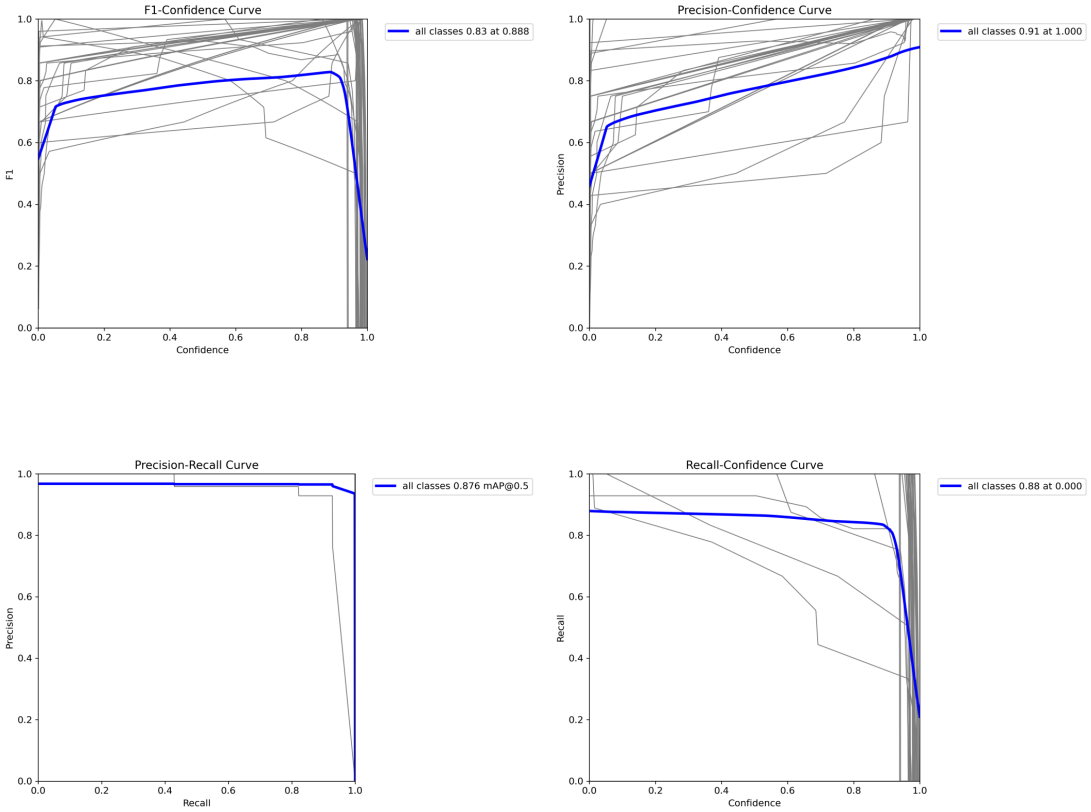


Figure 4. YOLOv8s Baseline 신뢰도 임계값 기반 탐지 성능 분석

(F1·Precision·Recall·PR Curve)

▼ Figure 4 설명

- **F1-Confidence Curve (Confidence Threshold별 F1 Score 변화)**
  - x축은 confidence threshold, y축은 F1입니다.
  - 파란선 기준 최적점이 F1=0.83 at conf=0.888로 표시됩니다.
  - 해석: 임계값을 너무 낮추면 오탐이 늘고, 너무 높이면 미탐이 늘어 F1이 떨어집니다. 현재 모델은 약 0.88~0.90 근처에서 정밀도-재현율 균형이 가장 좋습니다.

- **Precision-Confidence Curve (Confidence Threshold별 Precision 변화)**
  - x축은 confidence threshold, y축은 precision입니다.

    - threshold를 올릴수록 precision이 전반적으로 상승하며,

      conf=1.0 근처에서 약 0.91까지 올라갑니다.
    - 해석: 확신이 높은 예측만 남길수록(conf를 높일수록) 틀린 검출(False Positive)이 줄어 precision은 좋아지지만, 대신 검출 개수가 줄어 recall이 떨어집니다.
    - confidence threshold를 올릴수록 정밀도(precision)가 상승하는 경향을 보여주었습니다. 이는 모델이 높은 신뢰도의 예측만 남길수록 오탐(False Positive)이 줄어든다는 의미입니다. 다만 threshold를 과도하게 높이면 검출 자체가 줄어 재현율(recall)이 함께 저하될 수 있습니다. 따라서 이 그래프는 "**정밀도 향상과 재현율 감소의 trade-off가 존재한다**"는 점을 보여주며, 실제 적용 시에는 **목적에 따라 threshold를 조정해야 한다**는 결론을 도출할 수 있습니다.
    - 균형 성능이 필요한 경우에는 F1 최대 지점(약 conf 0.888 부근)을 기준값으로 사용하는 것이 합리적입니다.

- **Precision-Recall Curve (Precision-Recall 관계 및 mAP@0.5)**
    - x축은 recall, y축은 precision이며,

      범례의 all classes 0.876 mAP@0.5는 이 곡선 면적(AP)의 평균값을 뜻합니다.
    - 해석: 대부분 구간에서 precision이 높게 유지되어 PR 곡선이 상단에 위치합니다. 즉 전반 탐지 품질이 좋고, mAP@0.5=0.876 수준으로 baseline 성능이 안정적입니다.

- **Recall-Confidence Curve (Confidence Threshold별 Recall 변화)**
    - x축은 confidence threshold, y축은 recall입니다.
    - conf=0에서 recall이 최대(0.88)이고, threshold가 올라갈수록 recall이 완만히 감소하다가 고임계값에서 급락합니다.
    - 해석: 임계값을 높이면(conf를 높일수록) 확실한 것만 정답을 인정하므로, 실제 있는 객체를 놓치는 검출 누락(False Negative)이 증가합니다. 운영 시 "객체를 놓치면 안 되는" 목적이면 conf를 너무 높이지 말아야 하며, 이 그래프상 급락 전 구간(대략 0.85 전후까지)을 비교적 안전한 운용 범위라고 볼 수 있습니다.

▼ **Validation set 평가 결과**

- eval_mAP50: **0.8761**
- eval_mAP50_95: **0.8656**
- eval_mAP75_95: **0.8551**

- eval_Precision: **0.8693**
- eval_Recall: **0.8357**

▼ **초기 Baseline 모델 Kaggle 제출 결과**

Public score = **0.96138**

모델 학습 성능 향상 History (Top3)

▼ **성능 향상 목표**

본, 프로젝트는 데이터전처리 - Baseline모델 구축 - 모델 학습 성능 향상 과정이 병렬적으로 이루어졌기 때문에, Baseline 모델 설정 후, 데이터 전처리 및 모델 학습 방식 개선이 전체적으로 동시에 이루어지면서 성능을 높여가는 방향으로 수행되었습니다. 최종 목표는 kaggle 평가 지표인 Public score을 최대한 끌어올릴 수 있는 case를 만들어내는 것으로, 유의미한 결과가 보여진 대표 case들의 개선 사항을 정리하고자 합니다.

1) Baseline 모델 1차 개선

▼ **개요**

본 모델의 데이터 사용 및 전처리 방식은 Baseline과 동일하게 진행되었으며,

모델 학습 조건을 변경하여 개선을 시도했습니다.

▼ **개선 목표 및 스토리**

YOLOv8s에서 YOLOv8m으로 모델을 업그레이드하고 입력 해상도를 1024로 확장하였으며, 증강 데이터가 Kaggle 성능 저하(domain gap)를 유발한다고 판단해 원본 185장만 사용하고 bbox 정밀도 중심의 최소 증강·2-Phase 학습 및 TTA 적용으로 성능을 최적화하였습니다.

▼ **주요 변경 사항**

▼ **추가 사항**

- **모델 크기**
  - 변경 내용: YOLOv8s → YOLOv8m
  - 효과: + 10% mAP
- **해상도**
  - 변경 내용: 768 → 1024
  - 효과: bbox 정밀도 향상
- **Optimizer**
  - 변경 내용: SGD → AdamW
  - 효과: 안정적 수렴

- **LR Scheduler**
  - 변경 내용: Linear → Cosine
  - 효과: 미세 조정 개선
- **학습 전략**
  - 변경 내용: 단일 → 2단계
  - 효과: +2% mAP
- **학습률**
  - 변경 내용: 0.01 → 0.00005
  - 효과: 픽셀 단위 조정

▼ **제거 사항**

- **Mosaic**
  - 변경: 1.0 → 0.0
  - 효과: bbox 왜곡 방지
- **MixUp**
  - 변경 내용: 기본 → 0.0
  - 효과: bbox 왜곡 방지
- **Flip LR/UD**
  - 변경 내용: 0.5 → 0.0
  - 효과: bbox 박스 보존
- **HSV**
  - 변경 내용: 기본 → 0.0
  - 효과: 원본 색상 유지

▼ **개선 결과 지표 요약**

mAP[0.5:0.95]: 0.952

Kaggle Public score : **0.96859**

2) 데이터 추가 전처리 및 Baseline 모델 2차 개선

▼ **개요**

본 모델은 제공받은 **기본 데이터 + 외부 수집 데이터**를 모두 사용하여 학습이 진행되었습니다.

데이터 전처리 단계 및 모델 학습 조건을 변경하여 개선을 시도했습니다.

▼ **개선 목표 및 스토리**

추가된 학습용 외부 데이터에서, bbox를 annotation 오류가 있는 case를 발견하였고, 정성적 평가를 진행해 오류를 탐지하여 제외할 이미지 리스트(총 208개)를 만들었습니다. 초반에는 코딩을 통해 제외할 리스트 선별하기를 수차례 시도하였으나, 의도한 대로 진행되지 않아 2만 여개 이미지를 직접 검수하였습니다.

제거 리스트 경로 : configs/resources/exclude_4444_208.txt

▼ **주요 변경 사항**

- **데이터 전처리**
  - 외부 수집 데이터 추가 사용
  - 외부 수집 데이터 중 제거할 이미지 선별 & Baseline 전처리 과정에 제거 로직 추가
- **모델 크기**
  - 변경 내용: YOLOv8s → YOLOv8m
  - 효과: 모델 용량 증가로 표현력 향상, 작은/복잡 객체에서 성능 상승 가능
- **학습 횟수 (epochs)**
  - 변경 내용: 100 → 60
  - 효과: 학습 시간 단축, 과적합 위험 감소 가능
- **조기 종료 (patience)**
  - 변경 내용: 30 → 12
  - 효과: 성능 개선 정체 시 더 빠르게 종료, 불필요한 학습 방지
- **해상도 (imgsz)**
  - 변경 내용: 768 → 1024
  - 효과: 작은 객체/경계 정밀도 향상 가능, bbox 품질 개선
- **데이터 로딩 (workers)**
  - 변경 내용: 4 → 8
  - 효과: 데이터 로딩 병목 완화로 GPU 활용률 개선 가능
- **재실행 허용 (exist_ok)**
  - 변경 내용: true → false
  - 효과: 동일 run 이름 덮어쓰기 방지로 실험 결과 보존, 실수로 결과 교체하는 위험 감소
- **LR Scheduler (cos_lr)**
  - 변경 내용: false → true
  - 효과: 학습 후반 학습률을 부드럽게 낮춰 미세 조정 개선, 안정적 수렴 가능
- **Mosaic 종료 시점 (close_mosaic)**
  - 변경 내용: 10 → 35

    - 효과: Mosaic 증강을 더 오래 유지 → 다양한 조합 학습 증가
- **초기 학습률 (lr0)**
    - 변경 내용: 0.01 → 0.0018
    - 효과: 더 보수적인 업데이트로 안정적 학습, 발산 위험 감소
- **최종 학습률 비율 (lrf)**
    - 변경 내용: 0.01 → 0.05
    - 효과: 최종 학습률이 상대적으로 높아져 과도한 수렴 방지, 일반화 개선 가능
- **Box loss 가중치 (box)**
    - 변경 내용: 7.5 → 9.0
    - 효과: 박스 회귀에 더 많은 비중을 둬 위치/크기 정밀도 향상 가능
- **DFL loss 가중치 (dfl)**
    - 변경 내용: 1.5 → 2.0
    - 효과: 박스 품질(분포 기반 회귀) 강화로 정밀한 위치 추정에 기여 가능
- **Mosaic 비율 (mosaic)**
    - 변경 내용: 1.0 → 0.05
    - 효과: 강한 합성 증강을 줄여 실제 데이터 분포에 더 가깝게 학습, 정밀도 향상 가능

▼ **개선 결과 지표 요약**

- mAP[0.5:0.95] : **0.98682** (epoch 49)
- Kaggle Public score : **0.99361**

3) 데이터 추가 전처리 및 Baseline 모델 3차 개선 (최종 제출본 1)

▼ **개요**

본 모델은 제공받은 **기본 데이터 + 외부 수집 데이터**를 모두 사용하여 학습이 진행되었으며, 앞서 2차 개선 시에 적용한 208개 이미지 제외를 동일하게 적용했습니다. 학습 셋에서 최종 학습한 클래스 수는 118개 이지만, 제출 csv 구성 시 후처리를 적용해 74개 카테고리만 통과시켜 제출했습니다.

▼ **개선 목표 및 스토리**

목표는 모델 성능인 mAP를 개선하는 것 뿐 아니라, 제출 포맷 안정성 (행 수, image_id당 최대 박스 수, 카테고리 필터) 을 함께 확보하는 것입니다. 그래서 baseline 대비 성능을 높이기 위해 데이터셋 / 학습전략 / 제출후처리를 동시에 조정했습니다. 학습은 imgsz를 고해상도로 설정하고, rect=True로 비율 왜곡을 줄여서 경계 품질을 강화했으며, 제출에서는 74개 타깃 카테고리 필터를 적용해 리더보드 기준과 제출 포맷 제약을 동시에 만족시키도록 설계했습니다.

▼ **주요 변경 사항**

- **데이터 전처리**

    - 변경: data가 baseline run용 data.yaml → data_excluded_4444.yaml
    - 효과: 제외 규칙 적용된 데이터셋으로 학습 데이터 품질/구성 재정의
- **모델 크기**
    - 변경: YOLOv8s → YOLOv8m
    - 효과: 표현력 증가, 복잡 샘플에서 성능 개선 여지
- **학습 횟수 (epochs)**
    - 변경: 100 → 80
    - 효과: 학습 시간 단축, 장기 과적합 리스크 완화
- **조기 종료 (patience)**
    - 변경: 30 → 12
    - 효과: 성능 정체 구간에서 더 빠른 종료, 불필요 학습 감소
- **입력 해상도 (imgsz)**
    - 변경: 768 → 1280
    - 효과: 작은 객체/경계 정보 보존 강화, 박스 정밀도 개선 기대 (알약 경계/각인 보존)
- **직사각형 학습 (rect)**
    - 변경: false → true
    - 효과: 종횡비 왜곡 완화, 형태 보존에 유리 (알약 경계/각인 왜곡 최소화)
- **데이터 로딩 (workers)**
    - 변경: 4 → 8
    - 효과: 로딩 병목 완화, GPU 유휴시간 감소 가능
- **LR 스케줄러 (cos_lr)**
    - 변경: false → true
    - 효과: 후반 학습률 완만 감소로 수렴 안정성 개선 기대 (대형 해상도에서 안정적 수렴 유도.)
- **초기/최종 LR (lr0/lrf)**
    - 변경: 0.01/0.01 → 0.0018/0.05
    - 효과: 초기 업데이트 보수화 + 후반 LR 바닥 상승으로 과도 수렴 방지
- **손실 가중치 (box/cls/dfl)**
    - 변경: 7.5/0.5/1.5 → 9.0/0.5/2.0
    - 효과: 위치 회귀/박스 품질 항목 비중 강화 (cls는 유지, 비중을 충분히 주되 분류 손실과 균형 유지.)

- **Mosaic (mosaic/close_mosaic)**
  - 변경: 1.0/10 → 0.0/0
  - 효과: 합성 증강 제거로 실제 분포 중심 학습, 경계 왜곡 리스크 감소

▼ **제출 생성 로직 변경사항 및 적용 파라미터**

- 제출은 4_submission.py에서 생성했으며, 가중치는 best.pt를 자동 선택하는 기본 로직을 따랐다. (competition_best.pt가 없어 best.pt 자동 선택)

▼ **적용 파라미터**

- **conf threshold**: 0.10 (submission_manifest.json의 conf_threshold)
  min-conf: 0.24 (파일명 규칙 ..._minc024.csv 기준)
  NMS IoU: 0.50 (기본값/운영값)
  max_det_per_image(topk): 4
  target74_category_ids.txt **기반 카테고리 필터** : 74개 (완전히 일치하는 것)

▼ **최종 제출 모델 1 학습 결과**

- mAP50-95(B): 0.98615 (최고 : epoch 48)
  mAP50(B): 0.98937
  Precision(B): 0.99189
  Recall(B): 0.99323
- 제출 결과 CSV는 총 3,229행, 842개 이미지 예측으로 구성되었고,

  이미지당 최대 박스 수는 4를 만족했다.
- 제출 검증 결과는 PASS이며 에러는 0건이었다.
- Kaggle Public score : **0.99402**

4) 데이터 추가 전처리 및 Baseline 모델 4차 개선 (최종 제출본 2)

▼ **개요**

기존 Baseline(고해상도 YOLOv8m) 성능을 유지하면서 저신뢰 예측 구간을 줄이기 위해 데이터·학습·제출 후처리를 통합 고도화했다. exclude_4444 기반 데이터에 하드케이스 증강(low8 focus)을 추가하고, no-mosaic/rect/저학습률 파인튜닝으로 bbox 안정성을 확보했다. 이후 제출 단계에서 저점 행 후보를 선별 교체해 최종 2개 제출본(원본/교체본)을 운영했다.

▼ **개선 목표 및 스토리**

핵심 목표는 "평균 성능 상승"보다 "취약 구간 보강 + 전체 안정성 유지"였다. 먼저 저성능 클래스 중심 하드케이스를 오프라인 증강해 분포를 보완하고, 과격한 증강을 배제한 보수형 FT로 박스 품질을 고정했다. 마지막으로 저신뢰 행만 조건부 교체하는 후처리 실험을 병행해 리스크를 통제했다. 결과적으로 안정형과 공격형을 동시에 확보해 제출 전략 유연성을 높였다.

▼ **주요 변경 사항**

- **데이터 전처리**

    - 변경: 기존 최종이전 버전의 data_excluded_4444.yaml 중심 학습에서, 최종본은 data_with_hardcase.yaml를 적용해 하드케이스(저성능 클래스 중심) 증강 데이터를 추가 반영
    - 효과: base 구성은 유지하면서 취약 클래스 난이도 샘플을 보강해 저점 구간 학습 안정성 강화 (train rows 8165 → 9833)
- **모델 시작점(FT 체인)**
    - 변경: 최종이전의 단일 best 기반 제출에서, 최종본은 best.pt를 시작점으로 v14 추가 파인튜닝 수행
    - 효과: 기존 성능 고점을 유지한 상태에서 저LR 미세조정으로 박스/분류 균형을 재정렬
- **학습 횟수 (epochs)**
    - 변경: 최종이전(베이스 80 중심) 대비, 최종본은 FT 단계에서 epochs=24 추가 운용
    - 효과: 급격한 업데이트 없이 고점 주변 성능을 추가 탐색
- **조기 종료 (patience)**
    - 변경: 최종이전 기준 12(베이스 운영)에서, 최종 FT(v14)에서는 patience=16으로 완화
    - 효과: 후반 진동 구간에서 성급 종료를 줄이고 안정 수렴 관찰 구간 확보
- **입력 해상도 (imgsz)**
    - 변경: 유지 (1280)
    - 효과: 경계/각인 정보 보존 전략을 최종본에서도 동일하게 유지
- **직사각형 학습 (rect)**
    - 변경: 유지 (true)
    - 효과: 종횡비 왜곡 억제 전략 지속, bbox 형태 안정성 유지
- **데이터 로딩 (workers)**
    - 변경: 최종 FT(v14)에서 workers=8 명시 유지
    - 효과: 고해상도 학습 시 입력 병목 최소화
- **LR 스케줄러 (cos_lr)**
    - 변경: 유지 (true)
    - 효과: FT 후반 학습률 완만 감소로 미세조정 안정성 확보
- **초기/최종 LR (lr0/lrf)**
    - 변경: 최종이전 베이스 0.0018/0.05 대비, 최종 FT(v14)는 0.00015/0.05로 추가 하향
    - 효과: 이미 수렴된 가중치의 과한 이동을 억제하고 고점 주변 미세 탐색에 최적화
- **손실 가중치 (box/cls/dfl)**
    - 변경: 유지 (9.0/0.5/2.0)

    - 효과: 기존 박스 품질 강화 기조를 유지하며 분류 균형 지속
- **Mosaic (mosaic/close_mosaic)**
    - 변경: 유지 (0.0/0)
    - 효과: 합성 왜곡 배제 전략을 최종본에서도 동일하게 적용
- **competition_best 선택 로직**
    - 변경: 최종본(v14)은 2_train.py 기반 자동 재평가 로직으로 best/last를 mAP75_95 기준 비교 후 competition_best.pt 생성
    - 효과: 단순 저장 best 의존을 줄이고 대회 지표 기준의 자동 선택 일관성 확보
    - 결과: v14에서는 best가 선택되어 competition_best.pt와 best.pt가 동일 선택 결과

▼ 제출 생성 로직 변경사항 및 적용 파라미터

- 변경: 제출 생성은 동일하게 4_submission.py 사용. 다만 최종본은 v14 가중치 체인(및 동일 파라미터)으로 재생성
- **적용 파라미터**
    - **conf threshold**: 0.10
    - **min-conf**: 0.24
    - **NMS IoU**: 0.50
    - **max_det_per_image(topk)**: 4
    - target74_category_ids.txt 기반 카테고리 필터: 74개 (일치 검증)
- **후처리(최종본에서 추가된 핵심 변경)**
    - 변경: v14 원본 제출본 기준 score 하위 10%(q10) 구간에서, 비교 제출본 대비 점수 우위 후보 109건을 선별해 bbox+score 일괄 교체한 swap109_q10 버전 추가 생성
    - 효과: 평균 score 상승(원본 대비 증가) 후보 확보, 원본/교체본 2트랙 최종 제출 전략 가능
- **최종 제출 운영**
    - 변경: 단일 제출 운영에서, 최종본은 안정형(원본) + 공격형(후처리 교체본) 병행 운영으로 전환
    - 효과: 점수 상향 시도와 리스크 분산을 동시에 달성하도록 제출 전략 고도화

▼ 최종 제출 모델 2 학습 결과

- mAP50-95 : 0.98682
- mAP50 : 0.99004
- Precision: 0.993806
- Recall: 0.996274

- Kaggle Public score : **0.99524**

4. 실험 결과 및 분석 (Evaluation)

▼ 모델 평가 과정 설계 (STAGE 3)

▼ STAGE 3: 3_evaluate.py에서 평가 진행

STAGE 3은 학습(STAGE 2)에서 만들어진 가중치를 **검증 데이터셋으로 다시 평가**해 정량 지표를 확정하고, 결과 산출물과 메타데이터를 표준 경로에 저장하는 단계입니다.

1) 입력 및 구성 로딩

- 평가 스크립트는 *.yaml을 _base_ 체인까지 병합해 실험 설정을 로드합니다. (base.yaml의 공통 설정도 모두 포함)
- CLI에서 --device를 지정하면 evaluate.device 값을 덮어쓰도록 되어 있어, 동일 설정이라도 평가 디바이스 (Ultralytics가 지원하는 장치 표기들) 를 유연하게 바꿀 수 있습니다.

2) 평가 대상 가중치 선택 로직

- 평가는 항상 runs/<run_name>/weights/ 아래에서 가중치를 찾아 진행합니다.
- 먼저 해당 폴더에 competition_best.pt (대회 지표 기준으로 다시 평가해서 더 좋은 쪽을 뽑아 별도 파일로 저장한 것) 가 있으면 이를 우선 사용하고, 없으면 best.pt를 사용합니다.

3) 평가 데이터셋 지정 방식

- 평가 데이터는 STAGE 1에서 생성된 YOLO 데이터셋을 기준으로 합니다.

  직접 data.yaml파일을 찾아서 평가 입력으로 사용합니다.

  즉, 평가 입력은 항상 파이프라인에서 생성된 데이터셋 정의 파일을 사용합니다.

4) 평가 실행과 산출물 관리

- 평가는 PillDetector.validate()를 통해 Ultralytics model.val()을 호출합니다.
- 산출물은 runs/<run_name>/eval/val에 저장되며, 동일 run_name으로 재평가할 때 이전 산출물이 섞이지 않도록 eval/val 디렉터리를 삭제 후 재생성합니다. 이 설계는 결과 재현성과 비교 용이성을 높일 수 있습니다.

5) 평가 지표 기록 체계

- 평가 결과는 metrics.json에 저장됩니다.
- 기존 metrics.json이 있으면 학습(STAGE 2) 지표를 유지한 채, 평가 지표를 eval_ 접두어로 병합해 기록합니다.
  - **학습** 지표: **mAP50**, **mAP50_95**
  - 평가 지표: eval_**mAP50**, eval_**mAP50_95**

- 이를 통해 학습 중 검증 지표와 평가 지표를 명확히 구분할 수 있습니다.
- 추가로 eval_artifacts_dir와 eval_artifacts_rel 경로도 함께 기록해 산출물 위치를 추적 가능하게 한다. **평가 지표와 산출물을 한 파일에서 동시에 추적**할 수 있는 구조를 만듭니다.

6) 레지스트리 업데이트

- 평가가 끝나면 _registry.csv에서 해당 run_name 행을 찾아 다음 항목을 갱신합니다.
  - best_map50
  - best_map50_95
  - best_map75_95 (있을 경우)
  - notes = eval_complete

7) 평가 종료 후 출력

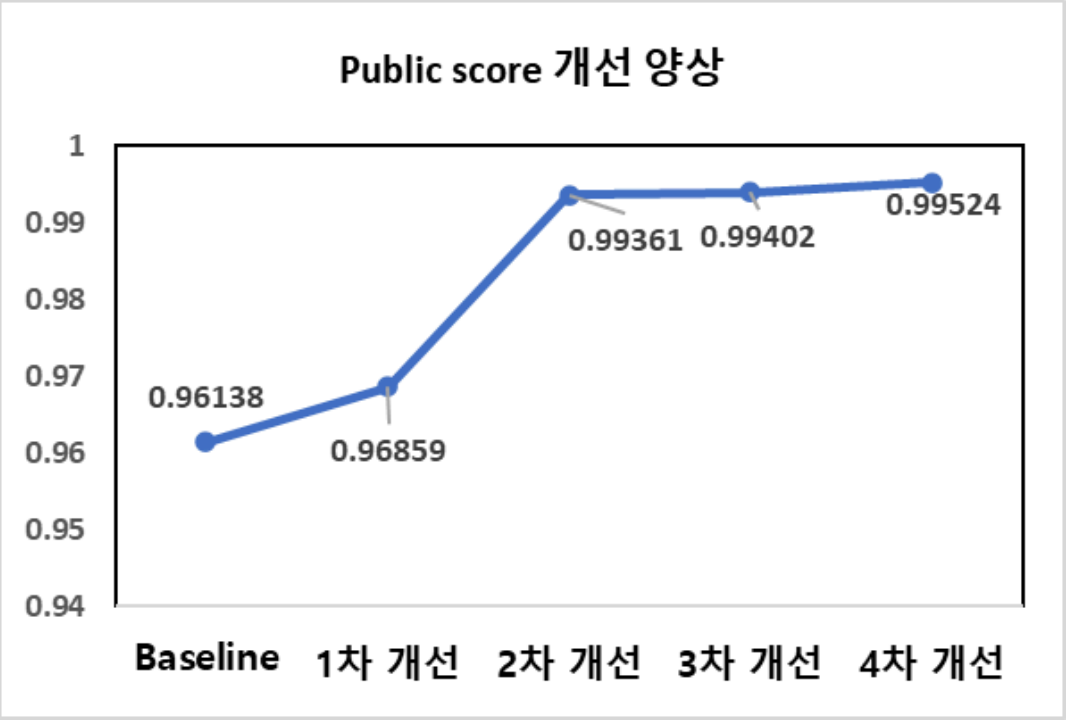
- 평가 종료 시 콘솔에 다음 지표를 요약 출력합니다.
  - eval_mAP50, eval_mAP50-95, eval_mAP75-95
  - precision, recall
- 클래스별 mAP75_95가 존재하면 클래스 개수 정보를 함께 제공하고, 20개 이하일 경우 클래스별 값을 출력합니다.

▼ **요약**

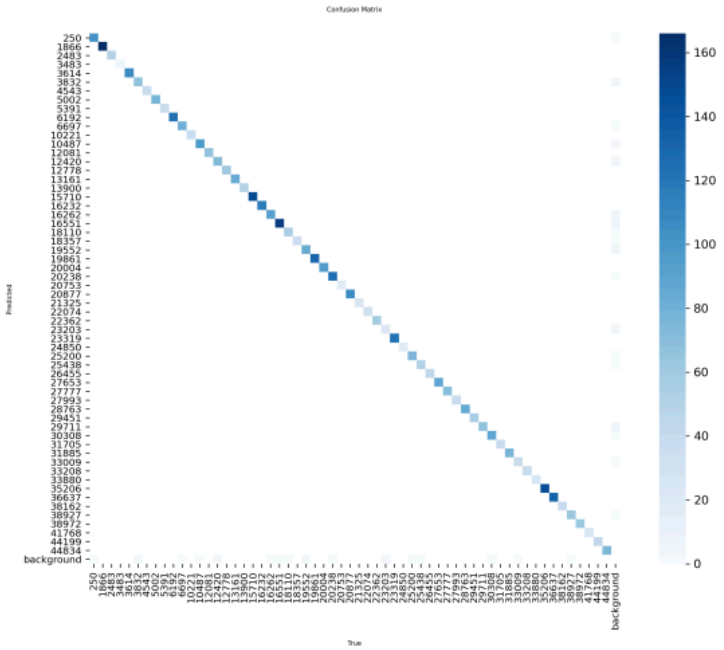
STAGE 3 평가는 STAGE 2 산출물(competition_best 또는 best)을 자동 선택해, STAGE 1의 data.yaml을 기준으로 검증을 수행하고, 결과를 eval 산출물과 metrics.json의 eval_ 지표로 체계적으로 저장하며 레지스트리를 갱신하도록 설계되어 있다.

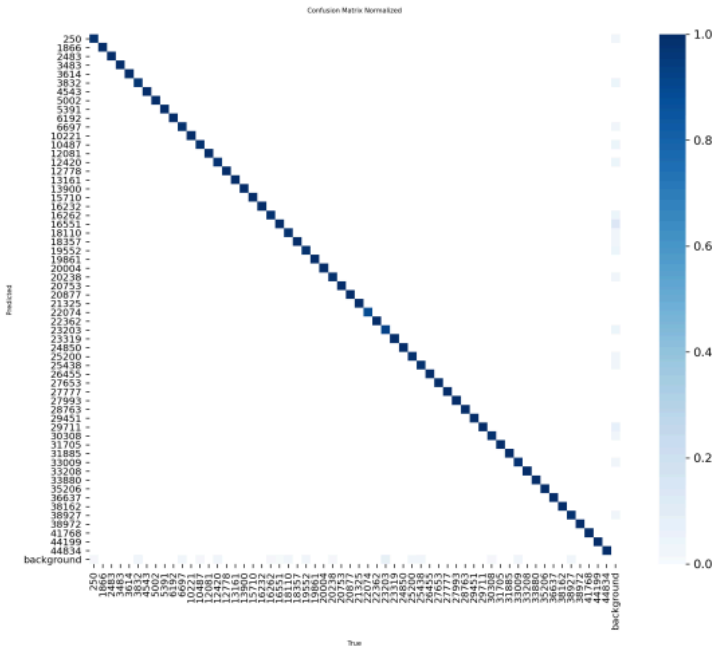
▼ 최종 제출 모델 학습 결과

1) 모델 학습 성능 개선 양상 요약



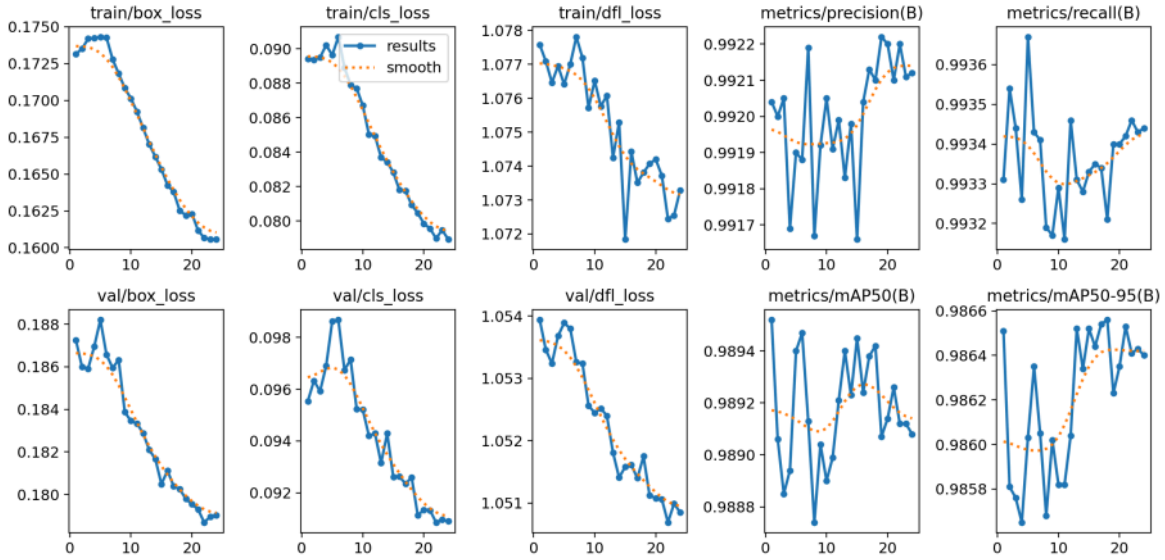
2) 학습 결과 시각화 (최종 제출 2 기준 : Kaggle public score = 0.99524)





▼ **Confusion matrix**

- 대각선(정답=예측)이 거의 진하게 이어져 있어서, 클래스 간 혼동이 전반적으로 낮습니다.
- 정규화 confusion matrix에서도 대각선이 거의 1.0에 가까워, 클래스별 상대 성능도 안정적입니다.
- 오른쪽 background 열/행에 약한 점들이 보이는데, 일부는 **미검출(FN) 또는 배경 오탐(FP)** 흔적으로 보입니다. 하지만 규모는 크지 않아 보입니다.

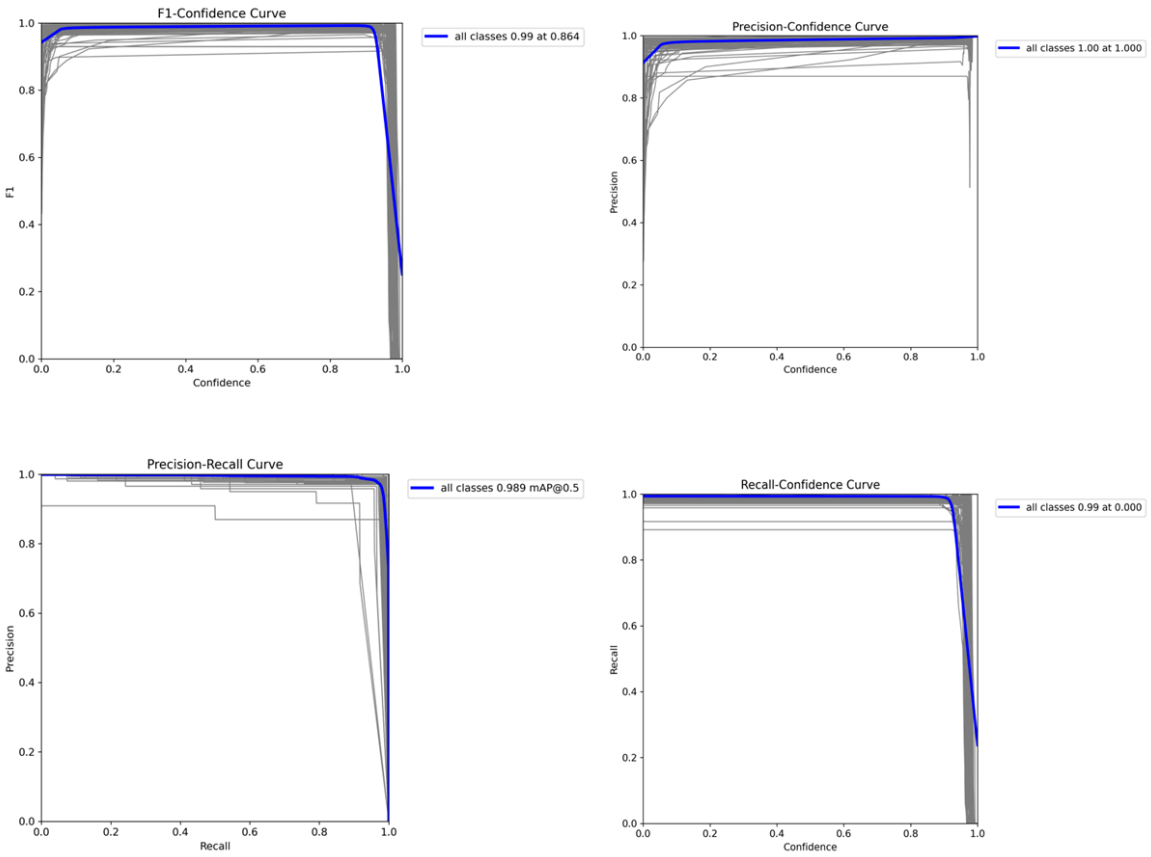


▼ **학습/검증 성능 추이 (Epoch별 Loss 및 Detection Metrics)**

학습이 안정적으로 수렴했고, 과적합 신호가 크지 않은 좋은 케이스로 보입니다.

- train/val의 box/cls/dfl loss가 같이 내려가서, 학습-검증 괴리가 크지 않습니다.
- precision, recall가 매우 높은 수준에서 흔들림만 있고 큰 붕괴가 없습니다.
- mAP50, mAP50-95도 후반에 유지/소폭 상승이라 성능이 안정화된 상태입니다.
- epoch 수가 많지 않은데도 지표가 이미 포화권이라, 추가 학습 이득은 크지 않을 가능성이 큽니다.

▼ 요약: **수렴 품질이 좋고 일반화도 양호한 결과**입니다.



▼ 신뢰도 임계값 기반 탐지 성능 분석 그래프

- **F1-Confidence**:

  최고점이 약 0.99 @ conf≈0.864로, 즉, F1 기준 최적 threshold가 꽤 높은 편입니다.

- **Precision-Confidence**

  conf가 올라갈수록 precision이 거의 1.0까지 유지오탐(FP)이 매우 낮다는 신호입니다.

- **Recall-Confidence**

  conf 낮은 구간에서 recall≈0.99, conf가 0.9 이후 급락threshold를 너무 높이면 미검출(FN)이 급증할 수 있습니다.

- **PR Curve**:

  면적이 매우 큼(mAP@0.5 ≈ 0.989)클래스 전반 분리력이 강한 상태입니다.

5. 결론 및 향후 과제 (Conclusion)

1) 프로젝트 요약

본 프로젝트는 YOLOv8m 기반 알약 검출 모델을 고도화하기 위해 데이터 정제, 하드케이스 오프라인 증강, 고해상도(rect) 파인튜닝, 제출 후처리를 통합 운영한 실험입니다. base 데이터에 저성능 클래스 중심 하드케이스를 추가하고, no-mosaic·저LR 조건으로 bbox 안정성을 확보했으며, 최종 제출은 저신뢰 구간 선택 교체 후처리까지 적용해 성능과 안정성을 함께 검증했습니다.

2) 한계점

1. Public 리더보드 기반 의사결정 비중이 높아 Private 분할에서 성능이 흔들릴 가능성이 있습니다.
2. 하드케이스 증강이 특정 취약 클래스에 집중되어 클래스 간 데이터 균형을 일부 왜곡할 가능성이 있습니다.
3. 고해상도(1280)·rect 기반 설정은 검출 품질에는 유리하나 학습/추론 자원 소모가 커 실서비스 적용 시 비용 부담이 큽니다.